Introduction

Welcome to the Patreon API!

Get familiar with the Patreon API and tools using the tutorials and references below.

Want more than a reference? For a broader overview and use cases, check our our developer portal.

We'd love to hear from you.

- For fastest technical help, our engineering team and community leaders are active on our forum at PatreonDevelopers.com.
- If you are an organization looking to partner with Patreon you can reach us at partnerships@patreon.com.
- If you are a third party developer looking to get your app added to our app directory, follow the instructions here.

Public API v1 is no longer maintained and will be deprecated soon. Please update your integrations to use API v2 and create a new v2 client on the Clients & API Keys page. For support, visit the Patreon Developers Forum.

Make these docs better

If you find any errors in our docs, we'd love to have you tell us! Share your feedback at patreondevelopers.com.

We are committed to ensuring continued access to the Patreon API. Existing API functionality will continue to be supported and maintained, and where needed, we will work to achieve and maintain parity with growing data and feature functionality in the Patreon product. Although we will continue to provide select support and ongoing maintenance of the API, we will have limited capacity in the near term to respond to questions through the developer forum. For any questions about how to best access or leverage the Patreon API, please refer to the following resources:

- Community support
- API Documentation
- App Directory

If you have an inquiry about partnering with Patreon, please contact us at partnerships@patreon.com.

Make sure to include a User-Agent header in your code that calls the API, otherwise your calls may be dropped with a 403 response. For the value of the User-Agent header, try to use an informative string that can identify your app, in a format like "MyCampaignName - Sync App", or "MyCampaignName - Website".

To be able to get your patrons' Discord user ids via the API, you must connect Patreon's Discord integration at your page at patreon.com and add it as a benefit to the tier(s) for which you will ask the Discord ids from the API. You can use your own Discord bot along Patreon's Discord integration.

API Libraries

We've written some open source libraries to help you use our platform services. Our API follows the JSON:API spec which is pretty complex and these libraries make it a bit easier to work with.

All of the libraries listed below require that you register a client application and get API keys.

Python

Get the package from PyPI, typically via pip:

INSTALL

```
pip install patreon
```

or

```
echo "patreon" >> requirements.txt
pip install -r requirements.txt
```

Make sure that, however you install patreon, you install its dependencies as well

View the source on github

PHP

Available on packagist

INSTALL

```
composer require patreon/patreon
```

View the source on github

Third Party Libraries

There are a number of third party libraries that the developer community has created.

Go

View the source on github

---

django-allauth

Patreon is available as an OAuth backend in django-allauth.

---

python-social-auth

Patreon is available as an OAuth backend in python-social-auth.

Clients and API Keys

In order to authenticate with OAuth and interact with the Patreon API, you'll have to register your Client(s). This involves signing up on patreon.com and making a creator account.

Once you've registered a Client you'll have access to a:

- **Client ID** – Used to identify your application/tool with the client you registered.
- **Client Secret** – Used to authenticate your application/tool with the client you registered.
- **Creator's Access Token** – Which can be used to access the API in the context of the creator you account you made when registering a client.
- **Creator's Refresh Token** – Can be used to refresh new access tokens.

> Note: Please never reveal your **Client Secret(s)**. If the secret is compromised, the attacker could get access to your campaign info, all of your patrons' profile info, email addresses, and their pledge amounts. If you fear your secret has been compromised, please let us know, and we will look into granting you a new id/secret pair.

OAuth

Patreon has an OAuth provider service — the technology that lets you log in to Medium with Twitter, log in to Disqus with Google+, and even login to Patreon with Facebook.

Below, you’ll find steps explaining how to begin integrating with us. It assumes understanding in HTTP protocol and OAuth, and that

you have administrative access & developer control of the server that you wish to integrate with Patreon.

All API calls must be made **via HTTPS**.

> Here are some helpful resources regarding these technologies: HTTP the Protocol Every Web Developer Must Know and An Introduction to OAuth 2

> Looking to dive in to the API? You can use your **Creator's Access Token** you get when registering a Client in place of the token you'd get back from the OAuth flow to start exploring the different endpoints or building a single creator application or tool.

Step 1 - Registering Your Client

To set up OAuth, you will need to register your client application on the Clients & API Keys page.

Step 2 - Making the Log In Button

Request [2]

```
GET www.patreon.com/oauth2/authorize
    ?response_type=code
    &client_id=<your client id>
    &redirect_uri=<one of your redirect_uris that you provided in step 1>
    &scope=<optional list of requested scopes>
    &state=<optional string>
```

Once your client is registered, you should create a “Log in with Patreon” and/or “Link your Patreon account” button on your site which directs users to the following URL:

HTTPS REQUEST

`GET www.patreon.com/oauth2/authorize`

QUERY PARAMETERS

Parameter	Description
response_type ***Required***	OAuth grant type. Set this to `code`.
client_id ***Required***	Your client id
redirect_uri ***Required***	One of your `redirect_uri`s that you provided in step 1

Parameter	Description
scope	This optional parameter will default to `users pledges-to-me my-campaign`, which fetches user profile information, pledges to your creator, and your creator info. It will be displayed to the user in human-friendly terms when signing in with Patreon. If your client requires the ability to ask for pledges or campaign data of **other users** (not just your own campaign), please contact us in the developers forum.
state	This optional parameter will be transparently appended as a query parameter when redirecting to your `redirect_uri`. This should be used as CSRF, and can be used as session/user identification as well. E.g. `https://www.patreon.com/oauth2/authorize?response_type=code&client_id=123&redirect_uri=https://www.mysite.com/custom-uri&state=their_session_id`. On this page, users will be asked if they wish to grant your client access to their account info. When they grant or deny access, **they will be redirected to the provided redirect_uri (so long as it is pre-registered with us)**.

Step 3 - Handling OAuth Redirect

Request [3]

```
GET https://www.mysite.com/custom-uri
    ?code=<single use code>
    &state=<string>
```

When the link in Step 2 redirects to the provided `redirect_uri`, e.g. https://www.mysite.com/custom-uri, it will bring extra HTTPS query parameters as follows (assuming the user granted your client access):

QUERY PARAMETERS

Parameter	Description
code	Used to fetch access tokens for the session that just signed in with Patreon.
state	Transparently appended from the state param you provided in your initial link in Step 2.

Note: Users that have not verified their email address through Patreon will receive an error message that they are required to verify their email address before they can use the OAuth flow. This verification email will be sent automatically to users upon receiving this message.

Step 4 - Validating Receipt of the OAuth Token

Request [4]

```
POST www.patreon.com/api/oauth2/token

code=<single use code, as passed in to GET route [2]>
&grant_type=authorization_code
&client_id=<your client id>
&client_secret=<your client secret>
&redirect_uri=<redirect_uri>
```

Make sure to also include this header: `Content-Type: application/x-www-form-urlencoded`

Your server should handle GET requests in Step 3 by performing the following request on the server (not as a redirect):

which will return a JSON response of:

```
{
    "access_token": <single use token>,
    "refresh_token": <single use token>,
    "expires_in": <token lifetime duration>,
    "scope": <token scopes>,
    "token_type": "Bearer"
}
```

to be stored on your server, one pair per user.

> Remember! - this step happens on your server. Our API Libraries typically handle this step for you or if you want to see examples of this step in other programming languages.

Step 5 - Using the OAuth Token

You may use the received `access_token` to make API calls. For example, a typical first usage of the new `access_token` would be to fetch the user's profile info, and either merge that into their existing account on your site, or make a new account for them. You could then use their pledge level to show or hide certain parts of your site.

> Remember! - this step happens on your server.

Step 6 - Resolving the OAuth Redirect

To reiterate, requests [4] and [5] should be performed by your server (synchronously or asynchronously) in response to receiving the GET request in request [3].

Once your calls are complete, you will have the user's profile info and pledge level for your creator.

If requests [4] and [5] were performed synchronously, then you can return a HTTPS 302 for their GET in request [3], redirecting to a page with appropriate success dialogs & profile information. If the requests in requests [4] and [5] are being performed asynchronously, your response to request [3] should probably contain AJAX code that will notify the user once requests [4] and [5] are completed.

Step 7 - Keeping up to date

Request [7]

```
POST www.patreon.com/api/oauth2/token

grant_type=refresh_token
&refresh_token=<the user's refresh_token>
&client_id=<your client id>
&client_secret=<your client secret>
```

which will return a JSON response of:

```
{
    "access_token": <single use token>,
    "refresh_token": <single use token>,
    "expires_in": <token lifetime duration>,
    "scope": <token scopes>,
    "token_type": "Bearer"
}
```

and you should store this information just as before.

Tokens are valid for the duration specified by "expires_in" field. During this period, you may refresh a user's information using the API calls from step 4. If you wish to get up-to-date information after the token has expired, a new token may be issued to be used. To refresh a token, make a POST request to the token endpoint with a grant type of `refresh_token`, as in the example. You may also manually refresh the token on the appropriate client in your clients page.

Errors

The Patreon API uses the following error codes:

Error Code	Meaning
400	Bad Request -- Something was wrong with your request (syntax, size too large, etc.)
401	Unauthorized -- Authentication failed (bad API key, invalid OAuth token, incorrect scopes, etc.)
403	Forbidden -- The requested is hidden for administrators only.
404	Not Found -- The specified resource could not be found.
405	Method Not Allowed -- You tried to access a resource with an invalid method.
406	Not Acceptable -- You requested a format that isn't json.
410	Gone -- The resource requested has been removed from our servers.
429	Too Many Requests -- Slow down!
500	Internal Server Error -- Our server ran into a problem while processing this request. Please try again later.
503	Service Unavailable -- We're temporarily offline for maintenance. Please try again later.

Rate Limits

To ensure the reliability, security, and performance of our platform, Patreon enforces rate limits on API usage. These limits help protect both our systems and your applications from unintentional misuse or abuse.

Client and Token Rate Limiting

Response example for client and token rate limits (HTTP 429)

```
{
  "errors": [
    {
      "code": null,
      "code_name": "RequestThrottled",
      "detail": "You have made too many attempts. Please try again later.",
      "id": "eb96e70d-909f-40cc-a3dc-e922cc90ea0a",
      "retry_after_seconds": 9,
      "status": "429",
      "title": "You have made too many attempts. Please try again later."
    }
  ]
}
```

API requests are subject to rate limits at both the client and access token levels:

- **Client:** Up to 100 requests every 2 seconds
- **Access Token:** Up to 100 requests per minute

Refreshing an access token does not reset or bypass the rate limits.

If your application exceeds these thresholds, it will receive an HTTP 429 Too Many Requests response. Applications should be designed to handle these gracefully, implementing retries with backoff as needed.

Responses for rate limited requests may optionally include the following response object. Treat all attributes in the response as optional. Use the `retry_after_seconds` value to determine when the next request can be made.

Edge Rate Limiting

Response example for edge rate limits (HTTP 429)

```
{
  "errors": [
    {
      "detail": "Too many bad requests. Time to cool down.",
      "retry_after_seconds": 1800
    }
  ]
}
```

A higher-level rate limit is enforced when a large number of bad requests (over 2,000 HTTP 4xx responses) are detected within a 10-minute window. Exceeding this threshold will result in a temporary block from the API for 30 minutes.

This is most commonly triggered by repeated:

- **HTTP 401 Unauthorized** (e.g., expired or invalid access tokens)
- **HTTP 429 Too Many Requests** (exceeding standard rate limits)

> Rate limits are subject to change, and applications should handle HTTP 429 responses gracefully.

APIv2: OAuth

WHAT'S NEW?

At a high level, the main differences between APIv1 and APIv2 are:

1. The Pledges resource has been replaced by the Members resource. Members return more data about the relationship between a patron and a creator, including charge status and membership lifetime.
2. **All data attributes and relationships must be explicitly requested** with the `fields` and `include` query params. In v1, the server would return certain attributes and relationships by default. Now, in v2, no data other than type and id will be returned for a resource unless it is requested.
3. The scopes have been improved. We have reworked what scopes are available in the API to provide better access for developers and better security for our users.
4. Developers can now create webhooks on campaigns on behalf of the creator, so your application can get real-time updates about a creator's campaign.

WHAT STAYS THE SAME?

1. Getting access to a Patreon user's account via OAuth works much the same. Just make sure to request all required scopes.
2. The client creator's access token will automatically have all V2 scopes associated with it.
3. We will not be deprecating APIv1 in the next year at least.

NOTE TO THOSE WITH V1 TOKENS:

You will be able to request tokens with any set of APIv2 scopes from your existing APIv1 client. If you choose to create an APIv2-specific client in the developer portal, that client will only be able to request V2 scopes.

Scopes

Scope	Description
identity	Provides read access to data about the user. See the /identity endpoint documentation for details about what data is available.
identity[email]	Provides read access to the user's email.
identity.memberships	Provides read access to the user's memberships.
campaigns	Provides read access to basic campaign data. See the /campaign endpoint documentation for details about what data is available.

Scope	Description
w:campaigns.webhook	Provides read, write, update, and delete access to the campaign's webhooks created by the client.
campaigns.lives	Allows reading of lives created by the campaign.
w:campaigns.lives	Allows creation and updating of lives on behalf of the campaign.
campaigns.members	Provides read access to data about a campaign's members. See the /members endpoint documentation for details about what data is available. Also allows the same information to be sent via webhooks created by your client.
campaigns.members[email]	Provides read access to the member's email. Also allows the same information to be sent via webhooks created by your client.
campaigns.members.address	Provides read access to the member's address, if an address was collected in the pledge flow. Also allows the same information to be sent via webhooks created by your client.
campaigns.posts	Provides read access to the posts on a campaign.

Note: During the OAuth2 authorization flow, Patreon currently *appends* newly requested scopes to any scopes a user has previously approved. Scopes are not overwritten or reduced.

As a result, reauthorizing with a smaller scope set will still grant your client the **union** of all previously approved scopes. This behavior may change in the future, so clients should always request the full set of scopes they require.

Using APIv2 with APIv1

During the transitionary period, when APIv1 and APIv2 are both available, it is possible to make requests against APIv1 endpoints with APIv2 clients and tokens. APIv2 has more controls around private information, so a fuller set of APIv2 scopes is needed to access V1 resources.

V1 Scope	V2 Scopes Required
campaigns	campaigns
my-campaign	campaigns
pledges	campaigns.members, campaigns.members[email], campaigns.members.address
pledges-to-me	identity.memberships
users	identity, identity[email]

APIv2: Resource Endpoints

All API requests should use the hostname https://www.patreon.com

With APIv2, all properties must be individually requested; there are no more default properties on resources. This is done by including the desired proprties in the fields parameter i.e.: `fields[address]=addressee,city`

GET /api/oauth2/v2/identity

Fetches the User resource.

Top-level `includes`: `memberships`, `campaign`.

This is the endpoint for accessing information about the current User with reference to the oauth token. With the basic scope of identity, you will receive the user's public profile information. If you have the `identity[email]` scope, you will also get the user's email address. You will not receive email address without that scope.

Fields for each include must be explicitly requested i.e. `fields[campaign]=summary,is_monthly&fields[user]=full_name,email` but url encode the brackets i.e. `fields%5Bcampaign%5D=summary,is_monthly&fields%5Buser%5D=full_name,email`

```
// Sample response with email scope for (url decoded) https://www.patreon.com/api/oauth2/v2/identity?fields[user]=about,created,email,first_name,full_nam
{
    "data": {
        "attributes": {
            "email": "some_email@email.com",
            "full_name": "Platform Team"
        },
        "id": "id",
        "relationships": {
            "campaign": {
                "data": {
                    "id": "id",
                    "type": "campaign"
                },
                "links": {
                    "related": "https://www.patreon.com/api/oauth2/v2/campaigns/id"
                }
            }
        },
        "type": "user"
    },
    "included": [
        {
            "attributes": {
                "is_monthly": true,
                "summary": "Hi There"
            },
            "id": "id",
            "type": "campaign"
        }
    ],
    "links": {
```

```
        "self": "https://www.patreon.com/api/oauth2/v2/user/id"
    }
}
```

You can request related data through includes, ie, `/api/oauth2/v2/identity?include=memberships` and `/api/oauth2/v2/identity?include=campaign`.

- If you request Campaign and have the campaigns scope, you will receive information about the user's Campaign.
- If you request Campaign and memberships, you will receive information about the user's memberships and the Campaigns they are Member of, provided you have the `campaigns` and `identity.memberships` scopes.
- If you request memberships and DON'T have the `identity.memberships` scope, you will receive data about the user's membership to your campaign. If you DO have the scope, you will receive data about all of the user's memberships, to all the campaigns they're members of.

GET /api/oauth2/v2/campaigns

Requires the `campaigns` scope. Returns a list of Campaigns owned by the authorized user.

Top-level `includes`: `tiers`, `creator`, `benefits`, `goals`.

Fields for each include must be explicitly requested i.e. `fields[tier]=currently_entitled_tiers` but url encode the brackets i.e. `fields%5Btier%5D=currently_entitled_tiers`

```
//Sample response for (url decoded) https://www.patreon.com/api/oauth2/v2/campaigns?fields[campaign]=created_at,creation_name,discord_server_id,image_sma
{
    "data": [
        {
            "attributes": {
                "created_at": "2018-05-04T23:34:08+00:00",
                "creation_name": "online communities",
                "discord_server_id": "1234567890",
                "google_analytics_id": "1234567890",
                "has_rss": true,
                "has_sent_rss_notify": true,
                "image_small_url": "https://example.url",
                "image_url": "https://example.url",
                "is_charged_immediately": false,
                "is_monthly": false,
                "is_nsfw": false,
                "main_video_embed": null,
                "main_video_url": "https://example.url",
                "one_liner": null,
                "patron_count": 2,
                "pay_per_name": "creation",
                "pledge_url": "/bePatron?c=1234560",
                "published_at": "2018-05-09T17:12:01+00:00",
                "rss_artwork_url": "https://example.url",
                "rss_feed_title": "My custom feed",
                "summary": "Putting the internet to work for creators.",
                "thanks_embed": null,
                "thanks_msg": null,
                "thanks_video_url": null
            },
            "id": "1234560",
            "type": "campaign"
        }
    ],
    "meta": {
```

```
        "pagination": {
            "total": 1
        }
    }
}
```

GET /api/oauth2/v2/campaigns/{campaign_id}

Requires the `campaigns` scope. The single resource endpoint returns information about a single Campaign, fetched by campaign ID.

Top-level `includes`: `tiers`, `creator`, `benefits`, `goals`.

Fields for each include must be explicitly requested i.e. `fields[campaign]=created_at,creation_name` but url encode the brackets i.e. `fields%5Bcampaign%5D=created_at,creation_name`

```
//Sample response for (url decoded) https://www.patreon.com/api/oauth2/v2/campaigns/{campaign_id}?fields[campaign]=created_at,creation_name,discord_serve
{
    "data":
        {
            "attributes": {
                "created_at": "2018-04-01T15:27:11+00:00",
                "creation_name": "online communities",
                "discord_server_id": "1234567890",
                "image_small_url": "https://example.url",
                "image_url": "https://example.url",
                "is_charged_immediately": false,
                "is_monthly": true,
                "main_video_embed": null,
                "main_video_url": null,
                "one_liner": null,
                "patron_count": 1000,
                "pay_per_name": "month",
                "pledge_url": "/bePatron?c=12345",
                "published_at": "2018-04-01T18:15:34+00:00",
                "summary": "The most creator-first API",
                "thanks_embed": "",
                "thanks_msg": null,
                "thanks_video_url": null,
            },
           "id": "12345",
           "type": "campaign"
        },
}
```

GET /api/oauth2/v2/campaigns/{campaign_id}/members

Gets the Members for a given Campaign. Requires the `campaigns.members` scope.

Top-level `includes`: `address` (requires `campaigns.members.address` scope), `campaign`, `currently_entitled_tiers`, `user`, `pledge_history`.

Fields for each include must be explicitly requested i.e. `fields[tier]=currently_entitled_tiers` but url encode the brackets i.e. `fields%5Btier%5D=currently_entitled_tiers`

We recommend using `currently_entitled_tiers` to see exactly what a Member is entitled to, either as an include on the members list or on the member get.

Returns from this endpoint have 1000 results in one page. If pledge events are requested in includes, it has 500 results in one page. Do make sure to paginate your results by using next pagination cursor and cycle until there is no next page cursor in the API return.

```
// Sample response for (url decoded) https://www.patreon.com/api/oauth2/v2/campaigns/{campaign_id}/members?include=currently_entitled_tiers,address&field
{
    "data": [
        {
            "attributes": {
                "full_name": "Platform Team",
                "is_follower": false,
                "last_charge_date": "2018-04-01T21:28:06+00:00",
                "last_charge_status": "Paid",
                "lifetime_support_cents": 400,
                "currently_entitled_amount_cents": 400,
                "patron_status": "active_patron",
            },
            "id": "03ca69c3-ebea-4b9a-8fac-e4a837873254",
            "relationships": {
                "address": {
                    "data": {
                        "id": "12345",
                        "type": "address"
                    }
                },
                "currently_entitled_tiers": {
                    "data": [{
                        "id": "54321",
                        "type": "tier",
                    }]
                }
            },
            "type": "member",
        },
        ...other members...,
    ],
    "included": [
        {
            "attributes": {
                "addressee": "Platform Team",
                "city": "San Francisco",
                "country": "US",
                "created_at": "2018-06-03T16:23:38+00:00",
                "line_1": "555 Main St",
                "line_2": "",
                "phone_number": null,
                "postal_code": "94103",
                "state": "CA"
            },
            "id": "12345",
            "type": "address"
        },{
        "attributes": {
            "amount_cents": 100,
            "created_at": "2018-04-01T04:15:41.403645+00:00",
            "description": "A tier",
            "discord_role_ids": ["1234567890"],
            "edited_at": "2018-04-01T02:55:36.963334+00:00",
            "patron_count": 32,
            "published": true,
            "published_at": "2018-04-01T02:55:36.938342+00:00",
            "requires_shipping": false,
            "title": "Patron",
```

```
            "url": "/bePatron?c=1231345&rid=54512321",
        },
        "id": "54321",
        "type": "tier",
    }],
    "meta": {
        "pagination": {
            "cursors": {
                "next": "12345678ab1231cdefg"
            },
            "total": 100
        }
    }
}
```

GET /api/oauth2/v2/members/{member_id}

Get a particular member by their id. Requires the `campaigns.members` scope. Make sure that you are using the member id (UUID) and not the user id.

Top-level `includes`: `address` (requires `campaign.members.address` scope), `campaign`, `currently_entitled_tiers`, `user`.

Fields for each include must be explicitly requested i.e. `fields[address]=line_1,city` but url encode the brackets i.e. `fields%5Baddress%5D=line_1,city`

We recommend using `currently_entitled_tiers` to see exactly what a member is entitled to, either as an include on the members list or on the member get.


```
// Sample response for (url decoded) https://www.patreon.com/api/oauth2/v2/members/{member_id}?fields[address]=line_1,line_2,addressee,postal_code,city&f
{
    "data": {
        "attributes": {
            "full_name": "first last",
            "is_follower": false,
            "last_charge_date": "2020-10-01T11:18:36.000+00:00"
        },
        "id": "123-456-789",
        "relationships": {
            "address": {
                "data": {
                    "id": "123",
                    "type": "address"
                },
                "links": {
                    "related": "https://www.patreon.com/api/oauth2/v2/address/123"
                }
            },
            "user": {
                "data": {
                    "id": "123",
                    "type": "user"
                },
                "links": {
                    "related": "https://www.patreon.com/api/oauth2/v2/user/123"
                }
            }
        },
        "type": "member"
    },
```

```
    "included": [
        {
            "attributes": {
                "addressee": "123",
                "city": "city",
                "line_1": "line 1",
                "line_2": "Apt. 101",
                "postal_code": "123"
            },
            "id": "123",
            "type": "address"
        },
        {
            "attributes": {},
            "id": "123",
            "type": "user"
        }
    ],
    "links": {
        "self": "https://www.patreon.com/api/oauth2/v2/members/123-456-789"
    }
}
```

GET /api/oauth2/v2/campaigns/{campaign_id}/posts

Get a list of all the Posts on a given Campaign by campaign ID. Requires the `campaigns.posts` scope.

GET /api/oauth2/v2/posts/{id}

Get a particular Post by ID. Requires the `campaigns.posts` scope.

POST /api/oauth2/v2/lives

> The Live APIs are early-access, and may be subject to change based on feedback from our partners. Please reach out if you are interested in using them.

Create a new Live livestream for a Campaign. Requires the `w:campaigns.lives` scope.

The `state` attribute must be either `pre_live` or `live`. If `state` is `pre_live`, the livestream is scheduled and `scheduled_for` must be provided. If `state` is `live`, the livestream starts immediately. Access rules can be provided to control who can view the livestream.

```
// Sample request body
{
  "data": {
    "type": "live",
    "attributes": {
      "title": "test",
```

```
      "description": "test description",
      "state": "pre_live",
      "scheduled_for": "",
      // Access rule IDs from the Campaign.live_access_rules relationship. Note that multiple can be included, for example to target multiple paid tiers,
      "live_access_rule_ids": [1234]
    }
  }
}
```

GET /api/oauth2/v2/lives/{id}

> The Live APIs are early-access, and may be subject to change based on feedback from our partners. Please reach out if you are interested in using them.

Get a particular Live livestream by ID. Requires the `campaigns.lives` scope.

Returns the current state and streaming information for the livestream.

```
// Sample response
{
  "data": {
    "id": "1235",
    "type": "live",
    "attributes": {
      "title": "",
      "description": "",
      "state": "pre_live",
      "rtmp_url": "",
      "stream_key": ""
    }
  }
}
```

PATCH /api/oauth2/v2/lives/{id}

> The Live APIs are early-access, and may be subject to change based on feedback from our partners. Please reach out if you are interested in using them.

Update a Live livestream's state. Requires the `w:campaigns.lives` scope.

Valid state transitions:

- From `pre_live`, can only go to `live`
- From `live`, can only go to `live_ended`
- From `live_ended`, no further transitions

```
// Sample request body
```

```
{
  "data": {
    "id": "1235",
    "type": "live",
    "attributes": {
      "state": "live"
    }
  }
}
```

APIv2: Webhook Endpoints

GET /api/oauth2/v2/webhooks

Get the Webhooks for the current user's Campaign created by the API client. You will only be able to see webhooks created by your client. Requires the `w:campaigns.webhook` scope.

Top-level `includes`: `client`, `campaign`.

```
// Sample response https://www.patreon.com/api/oauth2/v2/webhooks/?fields[webhook]=last_attempted_at,num_consecutive_times_failed,paused,secret,triggers,
{
    "data": [
        {
            "attributes": {
                "last_attempted_at": "2018-04-01T20:09:18+00:00",
                "num_consecutive_times_failed": 0,
                "paused": false,
                "secret": "hereisaverycomplexsecret",
                "triggers": [
                    "members:create",
                    "members:delete",
                    "members:update"
                ],
                "uri": "https://requestb.in/123456"},
            "id": "2793",
            "type": "webhook",
        },
    ],
}
```

POST /api/oauth2/v2/webhooks

Create a Webhook on the current user’s campaign. Requires the `w:campaigns.webhook` scope.

PATCH /api/oauth2/v2/webhooks/{id}

Update a webhook with the given id, if the webhook was created by your client. Requires the `w:campaigns.webhook` scope.

- NOTE: If and only if `num_consecutive_times_failed` > 0, you have unsent events due to your webhook being unreachable on our last attempt. To send all your queued events, you can `PATCH /api/oauth2/v2/webhooks/{webhook_id}` with attribute `paused : false`. We'll attempt to send you all unsent events and report back with your client's response to us.

```
// Sample PATCH payload
{
  "data": {
    "id": "1234567",
    "type": "webhook",
    "attributes": {
      "triggers": ["members:create", "members:delete"],
      "uri": "https://www.example2.com",
      "paused": "false" // <- do this if you're attempting to send missed events, see NOTE in Example Webhook Payload
    }
  }
}
```

DELETE /api/oauth2/v2/webhooks/{id}

Delete a webhook with the given id. Requires the `w:campaigns.webhook` scope.

Triggers v2

Trigger	Cause
members:create	Triggered when a new member is created. Note that you may get more than one of these per patron if they delete and renew their membership. Member creation only occurs if there was no prior payment between patron and creator.
members:update	Triggered when the membership information is changed. Includes updates on payment charging events.
members:delete	Triggered when a membership is deleted. Note that you may get more than one of these per patron if they delete and renew their membership. Deletion only occurs if no prior payment happened, otherwise pledge deletion is an update to member status.
members:pledge:create	Triggered when a new pledge is created for a member. Does not trigger when a user becomes a free member or redeems a gift.
members:pledge:update	Triggered when a member updates (upgrade, downgrade) their pledge.
members:pledge:delete	Triggered when a member deletes their pledge. Does not trigger when a user cancels a free or gifted membership.
posts:publish	Triggered when a post is published on a campaign.
posts:update	Triggered when a post is updated on a campaign.
posts:delete	Triggered when a post is deleted on a campaign.

Note: When the webhooks API was made available in a limited beta to API v1 customers, we allowed triggers on the pledge model. In API v2, pledge has been deprecated and member is the resource of record. Any existing webhooks on pledge will continue to work until API v1 is deprecated.

```
// Example POST Payload
{
  "data": {
    "type": "webhook",
    "attributes": {
      "triggers": ["members:create", "members:update", "members:delete"],
      "uri": "https://www.example.com",
    },
    "relationships": {
      "campaign": {
        "data": {"type": "campaign", "id": "12345"},
      },
    },
  },
}

// Example API response
{
   "data":{
      "attributes":{
         "last_attempted_at": null,
         "paused":false,
         "num_consecutive_times_failed":0,
         "secret":"abcdefghiklmnopqrstuvwyz",
         "triggers":[
            "members:create",
            "members:update",
            "members:delete"
         ],
         "uri":"https://example.com/hooks/patreon"
      },
      "id":"3955",
      "type":"webhook"
   }
}
```

Webhook Responses

When a webhook fires, the data will look something like this. Note that there will be a `X-Patreon-Signature` header, which is the HEX digest of the message body HMAC signed (with MD5) using your webhook's secret. We suggest you use this to verify authenticity of the webhook event. Webhook secrets should not be shared.

```
{
  "data": {
    "attributes": {
      "currently_entitled_amount_cents": null,
      "full_name": "Platform",
      "is_follower": true,
      "last_charge_date": null,
      "last_charge_status": null,
      "lifetime_support_cents": 0,
```

```
      "note": "",
      "patron_status": null,
      "pledge_relationship_start": null
    },
    "id": "d485d5ac-6c82-42c6-9c08-c50cf01b73d7",
    "relationships": {
      "address": {
        "data": null
      },
      "campaign": {
        "data": {
          "id": "123456",
          "type": "campaign"
        }
      },
      "currently_entitled_tiers": {
        "data": []
      },
      "user": {
        "data": {
          "id": "987654321",
          "type": "user"
        }
      }
    },
    "type": "member"
  },
  "included": [
    ...campaign data...
    ...user data...
  ]
}
```


APIv2: Resources

Access Rule

An access rule that determines who can view a piece of content.

ACCESS RULE ATTRIBUTES

Attribute	Type	Description
access_rule_type	string	The type of access being granted. example: tier, patron, public

ACCESS RULE RELATIONSHIPS

Relationship	Type	Description
tier	Tier	

Address

A patron's shipping address.

ADDRESS ATTRIBUTES

Attribute	Type	Description
addressee	string	Full recipient name. Can be null.
city	string	City.
country	string	Country.
created_at	string (UTC ISO format)	Datetime address was first created.
line_1	string	First line of street address. Can be null.
line_2	string	Second line of street address. Can be null.
phone_number	string	Telephone number. Specified for non-US addresses. Can be null.
postal_code	string	Postal or zip code. Can be null.
state	string	State or province name. Can be null.

ADDRESS RELATIONSHIPS

Relationship	Type	Description
campaigns	array[Campaign]	The campaigns that have access to the address.
user	User	The user this address belongs to.

Benefit

A benefit added to the campaign, which can be added to a tier to be delivered to the patron.

BENEFIT ATTRIBUTES

Attribute	Type	Description
app_external_id	string	The third-party external ID this reward is associated with, if any. Can be null.
app_meta	object	Any metadata the third-party app included with this benefit on creation. Can be null.
benefit_type	string	Type of benefit, such as `custom` for creator-defined benefits. Can be null.
created_at	string (UTC ISO format)	Datetime this benefit was created.
deliverables_due_today_count	integer	Number of deliverables for this benefit that are due today specifically.

Attribute	Type	Description
delivered_deliverables_count	integer	Number of deliverables for this benefit that have been marked complete.
description	string	Benefit display description. Can be null.
is_deleted	boolean	`true` if this benefit has been deleted.
is_ended	boolean	`true` if this benefit is no longer available to new patrons
is_published	boolean	`true` if this benefit is ready to be fulfilled to patrons.
next_deliverable_due_date	string (UTC ISO format)	The next due date (after EOD today) for this benefit. Can be null.
not_delivered_deliverables_count	integer	Number of deliverables for this benefit that are due, for all dates.
rule_type	string	A rule type designation, such as `eom_monthly` or `one_time_immediate`. Can be null.
tiers_count	integer	Number of tiers containing this benefit.
title	string	Benefit display title.

BENEFIT RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	The Campaign the benefit belongs to
campaign_installation	Campaign-Installation	(Alpha) The campaign-app connection
deliverables	array[Deliverable]	The Deliverables that have been generated by the Benefit
tiers	array[Tier]	The Tiers the benefit has been added to.

Campaign v2

The creator's page, and the top-level object for accessing lists of members, tiers, etc.

CAMPAIGN V2 ATTRIBUTES

Attribute	Type	Description
created_at	string (UTC ISO format)	Datetime that the creator first began the campaign creation process. See `published_at`.
creation_name	string	The type of content the creator is creating, as in "`vanity` is creating `creation_name`". Can be null.
currency	string	The campaign's currency.

Attribute	Type	Description
discord_server_id	string	The ID of the external discord server that is linked to this campaign. Can be null.
google_analytics_id	string	The ID of the Google Analytics tracker that the creator wants metrics to be sent to. Can be null.
has_rss	boolean	Whether this user has opted-in to rss feeds.
has_sent_rss_notify	boolean	Whether or not the creator has sent a one-time rss notification email.
image_small_url	string	URL for the campaign's profile image.
image_url	string	Banner image URL for the campaign.
is_charged_immediately	boolean	`true` if the campaign charges upfront, `false` otherwise. Can be null.
is_eligible_for_live	boolean	Whether the campaign is elibible for live.
is_monthly	boolean	`true` if the campaign charges per month, `false` if the campaign charges per-post.
is_nsfw	boolean	`true` if the creator has marked the campaign as containing nsfw content.
main_video_embed	string	Can be null.
main_video_url	string	Can be null.
name	string	The campaign's display name.
one_liner	string	Pithy one-liner for this campaign, displayed on the creator page. Can be null.
patron_count	integer	Number of patrons having access to this creator.
pay_per_name	string	The thing which patrons are paying per, as in "`vanity` is making $1000 per `pay_per_name`". Can be null.
pledge_url	string	Relative (to patreon.com) URL for the pledge checkout flow for this campaign.
published_at	string (UTC ISO format)	Datetime that the creator most recently published (made publicly visible) the campaign. Can be null.
rss_artwork_url	string	The url for the rss album artwork. Can be null.
rss_feed_title	string	The title of the campaigns rss feed.
show_earnings	boolean	Whether the campaign's total earnings are shown publicly
summary	string	The creator's summary of their campaign. Can be null.
thanks_embed	string	Can be null.
thanks_msg	string	Thank you message shown to patrons after they pledge to this campaign. Can be null.
thanks_video_url	string	URL for the video shown to patrons after they pledge to this campaign. Can be null.

Attribute	Type	Description
url	string	A URL to access this campaign on patreon.com
vanity	string	The campaign's vanity. Can be null.

CAMPAIGN V2 RELATIONSHIPS

Relationship	Type	Description
benefits	array[Benefit]	The campaign's benefits.
campaign_installations	array[Campaign-Installation]	
categories	array[Category]	The campaign's categories.
creator	User	The campaign owner.
goals	array[Goal]	[DEPRECATED!] The campaign's goals. Will always be empty.
live_access_rules	array[Access Rule]	The access rules that can be used to create live events.
tiers	array[Tier]	The campaign's tiers.

Deliverable

The record of whether or not a patron has been delivered the benefitthey are owed because of their member tier.

DELIVERABLE ATTRIBUTES

Attribute	Type	Description
completed_at	string (UTC ISO format)	When the creator marked the deliverable as completed or fulfilled to the patron. Can be null.
delivery_status	string	One of `delivered`, `not_delivered`, `wont_deliver`.
due_at	string (UTC ISO format)	When the deliverable is due to the patron.

DELIVERABLE RELATIONSHIPS

Relationship	Type	Description
benefit	Benefit	The Benefit the Deliverables were generated for.
campaign	Campaign	The Campaign the Deliverables were generated for.
member	Member	The member who has been granted the deliverable.
user	User	The user who has been granted the deliverable. This user is the same as the member user.

Goal

A funding goal in USD set by a creator on a campaign.

GOAL ATTRIBUTES

Attribute	Type	Description
amount_cents	integer	Goal amount in USD cents.
completed_percentage	integer	Equal to (pledge_sum/goal amount)*100, helpful when a creator wants to hide their pledge sum but still wants their goals to be cash based.
created_at	string (UTC ISO format)	When the goal was created for the campaign.
description	string	Goal description. Can be null.
reached_at	string (UTC ISO format)	When the campaign reached the goal. Can be null.
title	string	Goal title.

GOAL RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	The campaign trying to reach the goal

Live

A livestream associated with a campaign post.

LIVE ATTRIBUTES

Attribute	Type	Description
description	string	The description of the livestream. Can be null.
rtmp_url	string	The RTMP URL for streaming. Can be null.
scheduled_for	string (UTC ISO format)	The scheduled start time of the livestream. Can be null.
state	string	The current state of the livestream.
stream_key	string	The stream key for RTMP streaming. Can be null.
title	string	The title of the livestream. Can be null.

LIVE RELATIONSHIPS

Relationship	Type	Description

Relationship	Type	Description
campaign	Campaign	The campaign that owns this livestream.
live_access_rules	array[Access Rule]	The access rules that control who can view this livestream.

Media

A file uploaded to patreon.com, usually an image.

MEDIA ATTRIBUTES

Attribute	Type	Description
created_at	string (UTC ISO format)	When the file was created.
download_url	string	The URL to download this media. Valid for 24 hours.
file_name	string	File name.
image_urls	object	The resized image URLs for this media. Valid for 2 weeks.
metadata	object	Metadata related to the file. Can be null.
mimetype	string	Mimetype of uploaded file, eg: "application/jpeg".
owner_id	string	Ownership id (See also `owner_type`).
owner_relationship	string	Ownership relationship type for multi-relationship medias.
owner_type	string	Type of the resource that owns the file.
size_bytes	integer	Size of file in bytes.
state	string	The state of the file.
upload_expires_at	string (UTC ISO format)	When the upload URL expires.
upload_parameters	object	All the parameters that have to be added to the upload form request.
upload_url	string	The URL to perform a POST request to in order to upload the media file.

Member

The record of a user's membership to a campaign. Remains consistent across months of pledging.

MEMBER ATTRIBUTES

Attribute	Type	Description

Attribute	Type	Description
campaign_lifetime_support_cents	integer	The total amount that the member has ever paid to the campaign in the campaign's currency. `0` if never paid.
currently_entitled_amount_cents	integer	The amount in cents that the member is entitled to.This includes a current pledge, or payment that covers the current payment period.
email	string	The member's email address. Requires the `campaigns.members[email]` scope. Members may restrict the sharing of their email address.
full_name	string	Full name of the member user.
is_follower	boolean	[DEPRECATED!] The user is not a pledging patron but has subscribed to updates about public posts. This will always be false, following has been replaced by free membership.
is_free_trial	boolean	The user is in a free trial period.
is_gifted	boolean	The user's membership is from a free gift
last_charge_date	string (UTC ISO format)	Datetime of last attempted charge. `null` if never charged. Can be null.
last_charge_status	string	The result of the last attempted charge. The only successful status is `Paid`. `null` if never charged. One of `Paid`, `Declined`, `Deleted`, `Pending`, `Refunded`, `Fraud`, `Refunded by Patreon`, `Other`, `Partially Refunded`, `Free Trial`, `Refund Pending`, `Refund Declined`. Can be null.
lifetime_support_cents	integer	[DEPRECATED!] Use campaign_lifetime_support_cents. The total amount that the member has ever paid to the campaign in the campaign's currency. `0` if never paid.
next_charge_date	string (UTC ISO format)	Datetime of next charge. `null` if annual pledge downgrade. Can be null.
note	string	The creator's notes on the member.
patron_status	string	One of `active_patron`, `declined_patron`, `former_patron`. A null value indicates the member has never pledged. Can be null.
pledge_cadence	string	Number of months between charges. Can be null.
pledge_relationship_start	string (UTC ISO format)	Datetime of beginning of most recent pledge chainfrom this member to the campaign. Pledge updates do not change this value. Can be null.
will_pay_amount_cents	integer	The amount in cents the user will pay at the next pay cycle.

MEMBER RELATIONSHIPS

Relationship	Type	Description
address	Address	The member's shipping address that they entered for the campaign.Requires the `campaign.members.address` scope.

Relationship	Type	Description
campaign	Campaign	The campaign that the membership is for.
currently_entitled_tiers	array[Tier]	The tiers that the member is entitled to. This includes a current pledge, or payment that covers the current payment period.
pledge_history	array[Pledge Event]	The pledge history of the member
user	User	The user who is pledging to the campaign.

OAuthClient

A client created by a developer, used for getting OAuth2 access tokens.

OAUTHCLIENT ATTRIBUTES

Attribute	Type	Description
author_name	string	The author name provided during client setup. Can be null.
category	string	
client_secret	string	The client's secret.
default_scopes	string	(Deprecated in APIv2) The client's default OAuth scopes for the authorization flow.
description	string	The description provided during client setup.
domain	string	The domain provided during client setup. Can be null.
icon_url	string	The URL of the icon used in the OAuth authorization flow. Can be null.
name	string	The name provided during client setup.
privacy_policy_url	string	The URL of the privacy policy provided during client setup. Can be null.
redirect_uris	string	The allowable redirect URIs for the OAuth authorization flow.
tos_url	string	The URL of the terms of service provided during client setup. Can be null.
version	integer	The Patreon API version the client is targeting.

OAUTHCLIENT RELATIONSHIPS

Relationship	Type	Description
apps	array[Platform App]	(Alpha) The apps that this client controls.
campaign	Campaign	The campaign of the user who created the OAuth Client.
creator_token	OAuth Token	The token of the user who created the client.

Relationship	Type	Description
user	User	The user who created the OAuth Client.

Pledge Event

The record of a pledging action taken by the user, or that action's failure.

PLEDGE EVENT ATTRIBUTES

Attribute	Type	Description
amount_cents	integer	Amount (in the currency in which the patron paid) of the underlying event.
currency_code	string	ISO code of the currency of the event.
date	string (UTC ISO format)	The date which this event occurred.
payment_status	string	Status of underlying payment. One of `Paid`, `Declined`, `Deleted`, `Pending`, `Refunded`, `Fraud`, `Refunded by Patreon`, `Other`, `Partially Refunded`, `Free Trial`, `Refund Pending`, `Refund Declined`
pledge_payment_status	string	The payment status of the pledge. One of `queued`, `pending`, `valid`, `declined`, `fraud`, `disabled`.
tier_id	string	Id of the tier associated with the pledge.
tier_title	string	Title of the reward tier associated with the pledge.
type	string	Event type. One of `pledge_start`, `pledge_upgrade`, `pledge_downgrade`, `pledge_delete`, `subscription`

PLEDGE EVENT RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	The campaign being pledged to.
patron	User	The pledging user
tier	Tier	The tier associated with this pledge event.

Post v2

Content posted by a creator on a campaign page.

POST V2 ATTRIBUTES

Attribute	Type	Description
app_id	integer	Platform app id. Can be null.
app_status	string	Processing status of the post. Can be null.
content	string	Post content, may include html tags. Can be null.
embed_data	object	An object containing embed data if media is embedded in the post, None if there is no embed.
embed_url	string	Embed media url. Can be null.
is_paid	boolean	True if the post incurs a bill as part of a pay-per-post campaign. Can be null.
is_public	boolean	True if the post is viewable by anyone, False if only patrons (or a subset of patrons) can view. Can be null.
published_at	string (UTC ISO format)	Datetime that the creator most recently published (made publicly visible) the post. Can be null.
tiers	collection	Only patrons of tiers with these IDs can view this post. Empty array if no tiers are assigned even if is_paid is true. Can be null.
title	string	Can be null.
url	string	A URL to access this post on patreon.com.

POST V2 RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	The campaign that the membership is for.
user	User	The author of the post.

Tier

A membership level on a campaign, which can have benefits attached to it.

TIER ATTRIBUTES

Attribute	Type	Description
amount_cents	integer	Monetary amount associated with this tier (in U.S. cents).
created_at	string (UTC ISO format)	Datetime this tier was created.
description	string	Tier display description.
discord_role_ids	collection	The discord role IDs granted by this tier. Can be null.

Attribute	Type	Description
edited_at	string (UTC ISO format)	Datetime tier was last modified.
image_url	string	Full qualified image URL associated with this tier. Can be null.
patron_count	integer	Number of patrons currently registered for this tier.
post_count	integer	Number of posts published to this tier. Can be null.
published	boolean	`true` if the tier is currently published.
published_at	string (UTC ISO format)	Datetime this tier was last published. Can be null.
remaining	integer	Remaining number of patrons who may subscribe, if there is a `user_limit`. Can be null.
requires_shipping	boolean	`true` if this tier requires a shipping address from patrons.
title	string	Tier display title.
unpublished_at	string (UTC ISO format)	Datetime tier was unpublished, while applicable. Can be null.
url	string	Fully qualified URL associated with this tier.
user_limit	integer	Maximum number of patrons this tier is limited to, if applicable. Can be null.

TIER RELATIONSHIPS

Relationship	Type	Description
benefits	array[Benefit]	The benefits attached to the tier, which are used for generating deliverables.
campaign	Campaign	The campaign the tier belongs to.
tier_image	Media	The image file associated with the tier.

User v2

The Patreon user, which can be both patron and creator.

USER V2 ATTRIBUTES

Attribute	Type	Description
about	string	The user's about text, which appears on their profile. Can be null.
can_see_nsfw	boolean	`true` if this user can view nsfw content. Can be null.

Attribute	Type	Description
created	string (UTC ISO format)	Datetime of this user's account creation.
email	string	The user's email address. Can only be fetched via a user token with the `identity.email` scope. If you are using a creator token, use the email from the Member resource instead.
first_name	string	First name. Can be null.
full_name	string	Combined first and last name.
hide_pledges	boolean	`true` if the user has chosen to keep private which creators they pledge to. Can be null.
image_url	string	The user's profile picture URL, scaled to width 400px.
is_creator	boolean	`true` if this user has an active campaign.
is_email_verified	boolean	`true` if the user has confirmed their email.
last_name	string	Last name. Can be null.
like_count	integer	How many posts this user has liked.
social_connections	object	Mapping from user's connected app names to external user id on the respective app.
thumb_url	string	The user's profile picture URL, scaled to a square of size 100x100px.
url	string	URL of this user's creator or patron profile.
vanity	string	[DEPRECATED!] Use campaign.vanity. The public "username" of the user. patreon.com/<vanity> goes to this user's creator page. Non-creator users might not have a vanity. Can be null.

USER V2 RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	
memberships	array[Member]	Usually a zero or one-element array with the user's membership to the token creator's campaign, if they are a member. With the `identity.memberships` scope, this returns memberships to ALL campaigns the user is a member of.

Webhook

Webhooks are fired based on events happening on a particular campaign.

WEBHOOK ATTRIBUTES

Attribute	Type	Description

Attribute	Type	Description
last_attempted_at	string (UTC ISO format)	Last date that the webhook was attempted or used.
num_consecutive_times_failed	integer	Number of times the webhook has failed consecutively, when in an error state.
paused	boolean	`true` if the webhook is paused as a result of repeated failed attempts to post to `uri`. Set to `false` to attempt to re-enable a previously failing webhook.
secret	string	Secret used to sign your webhook message body, so you can validate authenticity upon receipt.
triggers	collection	List of events that will trigger this webhook.
uri	string	Fully qualified uri where webhook will be sent (e.g. https://www.example.com/webhooks/incoming).

WEBHOOK RELATIONSHIPS

Relationship	Type	Description
campaign	Campaign	The campaign whose events trigger the webhook.
client	OAuth Client	The client which created the webhook

Migrating from API v1 to API v2

API v1 is being deprecated. We strongly recommend migrating all integrations to API v2. Deprecation timeline details will be announced soon.

This guide covers the key differences between API v1 and API v2 to help you plan your migration.

Overview of Changes

API v2 introduces a redesigned resource model, new OAuth scopes, updated endpoints, and expanded webhook support. It also exposes new product features such as gifted memberships, lives, and free trials. Below is a summary of the main areas affected.

Endpoint Changes

v2 endpoints live under `/api/oauth2/v2/` instead of `/api/oauth2/api/`.

v1 Endpoint	v2 Equivalent
`/api/oauth2/api/current_user`	`/api/oauth2/v2/identity`
`/api/oauth2/api/campaigns/{campaign_id}/pledges`	`/api/oauth2/v2/campaigns/{campaign_id}/members`

See APIv2: Resource Endpoints for the full list.

Pledges to Members

The `pledge` resource has been replaced by the `member` resource. Members provide richer data about the patron-creator relationship including:

- `patron_status` — `active_patron`, `declined_patron`, or `former_patron`
- `last_charge_date` and `last_charge_status`
- `campaign_lifetime_support_cents`
- `currently_entitled_amount_cents`

See the Member resource for the full list of available attributes.

Explicit Field Requests

In v1, resources return certain attributes by default. In v2, **no attributes are returned unless explicitly requested** using `fields` and `include` query parameters.

For example:

```
GET /api/oauth2/v2/campaigns/{campaign_id}/members?fields[member]=full_name,patron_status,email&fields[user]=first_name
```

Note: brackets must be URL-encoded (e.g. `fields%5Bmember%5D`).

OAuth Scope Changes

v2 uses a new set of more granular OAuth scopes. If you are creating a new v2 client, you will need to request the appropriate v2 scopes.

v1 Scope	v2 Scope(s)
`users`	`identity`, `identity[email]`
`pledges-to-me`	`identity.memberships`
`my-campaign`	`campaigns`
`pledges`	`campaigns.members`, `campaigns.members[email]`, `campaigns.members.address`

v1 Scope	v2 Scope(s)
`campaigns`	`campaigns`

See APIv2: OAuth for the full scope reference.

Webhook Changes

Webhook triggers have been updated to reflect the new resource model.

v1 Trigger	v2 Trigger(s)
`pledge:create`	`members:pledge:create`
`pledge:update`	`members:pledge:update`
`pledge:delete`	`members:pledge:delete`

v2 also adds new triggers: `members:create`, `members:update`, `members:delete`, `posts:publish`, `posts:update`, and `posts:delete`.

Webhooks can now be managed programmatically via the Webhooks API.

New Resources in v2

v2 introduces several new resources:

- **Benefit** — creator-defined benefits attached to tiers
- **Deliverable** — tracks benefit fulfillment for members
- **Tier** — membership levels with associated benefits
- **Address** — patron shipping address data

Deprecated in v2

- **Goals** resource — always returns empty
- `is_follower` field on Member — the follower concept has been replaced by free membership (i.e. users with a membership to the free tier)
- `vanity` on User — use `campaign.vanity` instead

Pagination Changes

v2 uses cursor-based pagination with `page[count]` and `page[cursor]` parameters. The next cursor is returned in `meta.pagination.cursors.next` in the response.

Getting Started With Migration

1. Create a new v2 client on the Clients & API Keys page
2. Update your OAuth scopes to use v2 equivalents
3. Update your API calls to use v2 endpoints and explicitly request fields
4. Update any webhook integrations to use v2 triggers

APIv1: Resources (deprecated)

Our JSON responses follow the JSON:API standard, with the following structure for our three main resources (users, campaigns, and pledges):

When requesting some of these resources in our API they will have sensible defaults for what attributes are included. To request optional attributes, e.g. `like_count` and `comment_count`, specify the `fields` parameter in the URL like `https://www.patreon.com/api/oauth2/api/current_user?fields[user]=like_count,comment_count`. For more information, see the JSON:API docs.

User

```
{
  "type": "user"
  "id": <string>
  "attributes": {
    "first_name": <string>
    "last_name": <string>
    "full_name": <string>
    "vanity": <string>
    "email": <string>
    "about": <string>
    "facebook_id": <string>
    "image_url": <string>
    "thumb_url": <string>
    "youtube": <string>
    "twitter": <string>
    "facebook": <string>
    "created": <date>
    "url": <string>
    // optional properties
    "like_count": <int>
    "comment_count": <int>
  }
  "relationships": {
    "campaign": ...<campaign>...
  }
}
```

Campaign

```
{
  "type": "campaign"
  "id": <string>
  "attributes": {
    "summary": <string>
    "creation_name": <string>
    "pay_per_name": <string>
    "one_liner": <string>
    "main_video_embed": <string>
    "main_video_url": <string>
    "image_small_url": <string>
    "image_url": <string>
    "thanks_video_url": <string>
    "thanks_embed": <string>
    "thanks_msg": <string>
    "is_monthly": <bool>
    "is_nsfw": <bool>
    "created_at": <date>
    "published_at": <date>
    "pledge_url": <string>
    "pledge_sum": <int>
    "patron_count": <int>
    "creation_count": <int>
    "outstanding_payment_amount_cents": <int>
  }
  "relationships": {
    "creator": ...<user>...
    "rewards": [ ...<reward>, <reward>, ... ]
    "goals": [ ...<goal>, <goal>, ... ]
    "pledges": [ ...<pledge>, <pledge>, ... ]
  }
}
```

Pledge

```
{
  "type": "pledge"
  "id": <string>
  "attributes": {
    "amount_cents": <int> // Amount cents in the currency used by the patron
    "created_at": <date>
    "currency": <string> // Currency code of the pledge event (USD, GBP, EUR etc.)
    "declined_since": <date>
    "patron_pays_fees": <bool>
    "pledge_cap_cents": <int>
    // optional properties
    "total_historical_amount_cents": <int>
    "is_paused": <bool>
    "status": <string> // The status of this pledge (valid, declined, pending, disabled)
    "has_shipping_address": <bool>
  }
  "relationships": {
    "patron": ...<user>...
    "reward": ...<reward>... // Tier associated with this pledge
```

```
    "creator": ...<user>...
    "address": ...<address>...
  }
}
```

`amount_cents` is based in the **patron** currency which may be different from the campaign and tier currency.

`declined_since` indicates the date of the most recent payment if it failed, or ňull`if the most recent payment succeeded. A pledge with a non-null `declined_since` should be treated as **invalid**.

`total_historical_amount_cents` indicates the lifetime value this patron has paid to the campaign.

APIv1: Endpoints (deprecated)

Public API v1 is no longer maintained and will be deprecated soon. Please update your integrations to use API v2 and create a new v2 client on the Clients & API Keys page. For support, visit the Patreon Developers Forum.

To use a given `access_token`, send it in the Authorization HTTPS Header as follows:

```
Authorization: Bearer <access_token>
```

The three endpoints below are accessed using an OAuth client `access_token` obtained from the OAuth section. Please go there first if you do not yet have one.

When performing an API request, the information you are allowed is determined by which `access_token` you are using. Please be sure to select your `access_token` appropriately. For example, **if someone has granted your OAuth client access to their profile information, and you try to fetch it using your own Creator's Access Token instead of the one created when they granted your client access, you will instead just get your own profile information.**

During the transition period from APIv1 to APIv2, it is possible to use v2 scopes on v1 endpoints. If you do this, note that you may have more scope than you had in v1, and so the set of data returned will be much greater.

Fetching a patron's profile info

```
import url from 'url'
import patreonAPI, { oauth as patreonOAuth } from 'patreon'

const CLIENT_ID = null     // Replace with your data
const CLIENT_SECRET = null // Replace with your data
const redirectURL = null   // Replace with your data
```

```
const patreonOAuthClient = patreonOAuth(CLIENT_ID, CLIENT_SECRET)

function handleOAuthRedirectRequest(request, response) {
    const oauthGrantCode = url.parse(request.url, true).query.code

    patreonOAuthClient
        .getTokens(oauthGrantCode, redirectURL)
        .then(tokensResponse => {
            const patreonAPIClient = patreonAPI(tokensResponse.access_token)
            return patreonAPIClient('/current_user')
        })
        .then(({ store }) => {
            response.end(store.findAll('user').map(user => user.serialize()))
        })
        .catch(err => {
            console.error('error!', err)
            response.end(err)
        })
}

//Get the raw json from the response. See for the expected format of the data
var patreon_response = patreon_client('/current_user').then(function(result) {
  user_store = result.store
  const data = result.rawJson
  /*  data.data will contain the current_user, but there might be more users returned and loaded into the store.
   *  Get the id of the requested user, and find it in the store
   */
  const myUserId = data.data.id
  creator = user_store.find('user', myUserId)
})
```

Response:


```
{
  "data": {
    "attributes": {
      "about": null,
      "created": "2017-10-20T21:36:23+00:00",
      "discord_id": null,
      "email": "corgi@example.com",
      "facebook": null,
      "facebook_id": null,
      "first_name": "Corgi",
      "full_name": "Corgi The Dev",
      "gender": 0,
      "has_password": true,
      "image_url": "https://c8.patreon.com/2/400/0000000",
      "is_deleted": false,
      "is_email_verified": false,
      "is_nuked": false,
      "is_suspended": false,
      "last_name": "The Dev",
      "social_connections": {
        "deviantart": null,
        "discord": null,
        "facebook": null,
        "reddit": null,
        "spotify": null,
        "twitch": null,
        "twitter": null,
        "youtube": null
      },
      "thumb_url": "https://c8.patreon.com/2/100/0000000",
      "twitch": null,
```

```
      "twitter": null,
      "url": "https://www.patreon.com/corgithedev",
      "vanity": "corgithedev",
      "youtube": null
    },
    "id": "0000000",
    "relationships": {
      "pledges": {
        "data": []
      }
    },
    "type": "user"
  },
  "links": {
    "self": "https://www.patreon.com/api/user/0000000"
  }
}
```


This endpoint returns a JSON representation of the user who granted your OAuth client an `access_token`. It is most typically used in the OAuth "Log in with Patreon flow" to create or update the patron's account info in your application.

The Patreon JS library uses a data store pattern for storing inflated objects from the returned results of API calls. In some cases, especially if you have been granted the scopes for being a multi-campaign client or are opted-in to some API beta programs, the JS client calling `/current_user` will fetch the current user's campaign, as well as all the patron users connected to that campaign.

This can result in the user store in the JS library having a larger list of users than expected for a call to `/current_user`, but the current user's `user` object will be in that list.

HTTPS REQUEST

`GET https://www.patreon.com/api/oauth2/api/current_user`

Remember — you must pass the correct `access_token` from the user.

Fetch a creator profile and campaign info

```
import patreonAPI from 'patreon'

const accessToken = null   // Replace with your creator access token

const patreonAPIClient = patreonAPI(accessToken)
patreonAPIClient('/current_user/campaigns')
    .then(({ store }) => {
        const user = store.findAll('user').map(user => user.serialize())
        console.log('user is', user)
        const campaign = store.findAll('campaign').map(campaign => campaign.serialize())
        console.log('campaign is', campaign)
    })
    .catch(err => {
        console.error('error!', err)
        response.end(err)
    })
```

Response:

```
{
  "data": [{
    "attributes": {
      "created_at": "2017-10-20T21:39:01+00:00",
      "creation_count": 0,
      "creation_name": "Documentation",
      "discord_server_id": null,
      "display_patron_goals": false,
      "earnings_visibility": "public",
      "image_small_url": null,
      "image_url": null,
      "is_charged_immediately": false,
      "is_monthly": false,
      "is_nsfw": false,
      "is_plural": false,
      "main_video_embed": null,
      "main_video_url": null,
      "one_liner": null,
      "outstanding_payment_amount_cents": 0,
      "patron_count": 0,
      "pay_per_name": null,
      "pledge_sum": 0,
      "pledge_url": "/bePatron?c=0000000",
      "published_at": "2017-10-20T21:49:31+00:00",
      "summary": null,
      "thanks_embed": null,
      "thanks_msg": null,
      "thanks_video_url": null
    },
    "id": "0000000",
    "relationships": {
      "creator": {
        "data": {
          "id": "1111111",
          "type": "user"
        },
        "links": {
          "related": "https://www.patreon.com/api/user/1111111"
        }
      },
      "goals": {
        "data": []
      },
      "rewards": {
        "data": [{
            "id": "-1",
            "type": "reward"
          },
          {
            "id": "0",
            "type": "reward"
          }
        ]
      }
    },
    "type": "campaign"
  }],
  "included": [{
      "attributes": {
        "about": null,
        "created": "2017-10-20T21:36:23+00:00",
        "discord_id": null,
        "email": "corgi@patreon.com",
        "facebook": null,
        "facebook_id": null,
```

```
      "first_name": "Corgi",
      "full_name": "Corgi The Dev",
      "gender": 0,
      "has_password": true,
      "image_url": "https://c8.patreon.com/2/400/1111111",
      "is_deleted": false,
      "is_email_verified": false,
      "is_nuked": false,
      "is_suspended": false,
      "last_name": "The Dev",
      "social_connections": {
        "deviantart": null,
        "discord": null,
        "facebook": null,
        "reddit": null,
        "spotify": null,
        "twitch": null,
        "twitter": null,
        "youtube": null
      },
      "thumb_url": "https://c8.patreon.com/2/100/1111111",
      "twitch": null,
      "twitter": null,
      "url": "https://www.patreon.com/drkthedev",
      "vanity": "drkthedev",
      "youtube": null
    },
    "id": "1111111",
    "relationships": {
      "campaign": {
        "data": {
          "id": "0000000",
          "type": "campaign"
        },
        "links": {
          "related": "https://www.patreon.com/api/campaigns/0000000"
        }
      }
    },
    "type": "user"
  },
  {
    "attributes": {
      "amount": 0,
      "amount_cents": 0,
      "created_at": null,
      "description": "Everyone",
      "id": "-1",
      "remaining": 0,
      "requires_shipping": false,
      "type": "reward",
      "url": null,
      "user_limit": null
    },
    "id": "-1",
    "relationships": {
      "creator": {
        "data": {
          "id": "1111111",
          "type": "user"
        },
        "links": {
          "related": "https://www.patreon.com/api/user/1111111"
        }
      }
    },
```

```
      "type": "reward"
    },
    {
      "attributes": {
        "amount": 1,
        "amount_cents": 1,
        "created_at": null,
        "description": "Patrons Only",
        "id": "0",
        "remaining": 0,
        "requires_shipping": false,
        "type": "reward",
        "url": null,
        "user_limit": null
      },
      "id": "0",
      "relationships": {
        "creator": {
          "data": {
            "id": "1111111",
            "type": "user"
          },
          "links": {
            "related": "https://www.patreon.com/api/user/1111111"
          }
        }
      },
      "type": "reward"
    }
  ]
}
```


This endpoint returns a JSON representation of the user's campaign, including its rewards and goals, and the pledges to it. If there are more than twenty pledges to the campaign, the first twenty will be returned, along with a link to the next page of pledges.

HTTPS REQUEST

`GET https://www.patreon.com/api/oauth2/api/current_user/campaigns`

QUERY PARAMETERS

Parameter	Default	Description
includes	`rewards,creator,goals,pledges`	You can pass this `rewards`, `creator`, `goals`, or `pledges`

Remember — you must pass a valid `access_token`.

Paging through a list of pledges

```
// TODO: get pagination example
```

Response:

```
{
```

```
    "data": [
        {
            "attributes": {
                "amount_cents": 100,
                "created_at": "2016-07-25T20:59:52+00:00",
                "declined_since": null,
                "patron_pays_fees": false,
                "pledge_cap_cents": null
            },
            "id": "2745627",
            "relationships": {
                "patron": {
                    "data": {
                        "id": "111111",
                        "type": "user"
                    },
                    "links": {
                        "related": "https://www.patreon.com/api/user/111111"
                    }
                },
                "reward": {
                    "data": {
                        "id": "222222",
                        "type": "reward"
                    },
                    "links": {
                        "related": "https://www.patreon.com/api/rewards/222222"
                    }
                }
            },
            "type": "pledge"
        }
    ],
    "included": [
        {
            "attributes": {
                "about": "sample about text",
                "created": "2015-01-15T07:25:51+00:00",
                "email": "foo@bar.com",
                "facebook": null,
                "first_name": "Foo",
                "full_name": "Foo Bar",
                "gender": 1,
                "image_url": "",
                "is_email_verified": true,
                "last_name": "Bar",
                "social_connections": {
                    "deviantart": null,
                    "discord": null,
                    "facebook": null,
                    "reddit": null,
                    "spotify": null,
                    "twitch": null,
                    "twitter": null,
                    "youtube": null
                },
                "thumb_url": "",
                "twitch": null,
                "twitter": "foo",
                "url": "https://www.patreon.com/foo",
                "vanity": "foo",
                "youtube": null
            },
            "id": "111111",
            "type": "user"
        },
```

```
        {
            "attributes": {
                "amount_cents": 100,
                "created_at": "2017-12-19T01:56:37.762679+00:00",
                "description": "",
                "discord_role_ids": None,
                "edited_at": "2017-12-19T01:56:37.762679+00:00",
                "image_url": None,
                "patron_count": 1,
                "post_count": None,
                "published": True,
                "published_at": "2017-12-19T01:56:37.762679+00:00",
                "remaining": None,
                "requires_shipping": False,
                "title": "",
                "unpublished_at": None,
                "url": "/bePatron?c=12345&rid=222222",
                "user_limit": None
            },
            "id": "222222",
            "type": "reward"
        }
    ],
    "links": {
        "first": "https://www.patreon.com/api/oauth2/api/campaigns/70261/pledges?page%5Bcount%5D=10",
        "next": "https://www.patreon.com/api/oauth2/api/campaigns/70261/pledges?page%5Bcount%5D=10&page%5Bcursor%5D=2017-08-21T20%3A16%3A49.258893%2B00%3
    },
    "meta": {
        "count": 18
    }
}
```


This endpoint returns a JSON list of pledges to the provided `campaign_id`. They are sorted by the date the pledge was made, and provide relationship references to the **user** who made each respective pledge, as well as the **reward tier** pledged to.

The API response will also contain a `links` field which may be used to fetch the next page of pledges, or go back to the first page.

> When you made a creator page to gain API access, behind the scenes a campaign resource was created. You can access this resource as described in Fetching your own profile and campaign info after authenticating via OAuth with your creator account to gain your `campaign_id`.

HTTPS REQUEST

`GET https://www.patreon.com/api/oauth2/api/campaigns/<campaign_id>/pledges?include=patron.null`

PAGING

> Remember — you must pass a valid `access_token`.

You may only fetch your own list of pledges or public pledges. If you would like to create an application which can manage many creator's campaigns, please contact us in the developers forum.

More Data, Pagination

Requesting specific data

Want to retrieve the patrons for your pledges, or the goals for a given campaign?

To retrieve specific attributes or relationships other than the defaults, you can pass `fields` and `include` parameters respectively, each being comma-separated lists of attributes or resources. You can see which attributes or relationships are requestable on a given resource in the resources section.

> For more information on requesting specific data, the JSONAPI documentation may be useful.

REQUESTING OPTIONAL ATTRIBUTES

To fetch an optional property (for example `total_historical_amount_cents` and `is_paused` on `pledge`), use the `fields` query params.

`GET https://www.patreon.com/api/oauth2/api/campaigns/<campaign_id>/pledges?fields[pledge]=total_historical_amount_cents,is_paused`

> A common gotcha is forgetting to URL-encode the square brackets. For instance, a raw cURL will only work if you replace `[` with `%5B` and `]` with `%5D`.

RESTRICTING INCLUDED RESOURCES

By default, fetching or including a resource will follow that resource's relationship tree as well.

- To fetch a resource without any relationships, set `include=null`.
- To include a resource `foo` but not its relationships, set `include=foo.null`.
- You can also extend this to multiple includes, e.g. `include=foo.null,bar.baz.null`.

For a more explicit approach, you may instead set `json-api-use-default-includes=false`, which will limit relationships to only those specifically included.

Pagination

Many Patreon API endpoints are paginated. Use `page[count]` queryparam to limit how many records are returned per request. Use the `meta.pagination.cursors.next` value from the response to fetch the next page.

HOW IT WORKS

First request:

```
curl --request GET \
  --url https://www.patreon.com/api/oauth2/v2/campaigns/$campaign_id/members?page%5Bcount%5D=5 \
  --header "Authorization: Bearer $access_token"
```

Example response:

```
{
  "data": {},
  "meta": {
    "pagination": {
```

```
      "total": 10,
      "cursors": {
        "next": "03:eyJ2IjoxLCJjIjoiNmY2YWMxMGUtYmU2NS00MTc5LWEyODktMzkwZGMxMTFhMTQ1IiwidCI6IiJ9:1V2dmthW5w5REEsle2scJmOm"
      }
    }
  }
}
```

Requesting next page:

```
curl --request GET \
  --url https://www.patreon.com/api/oauth2/v2/campaigns/$campaign_id/members?page%5Bcount%5D=5&page%5Bcursor%5D=03:eyj2ijoxlcjjijoinmy2ywmxmgutymu2ns00mt
  --header "Authorization: Bearer $access_token"
```

```
{
  "data": {},
  "meta": {
    "pagination": {
      "total": 10,
      "cursors": {
        "next": null
      }
    }
  }
}
```

1. Request the first page
   - Specify how many records you want with `page[count]` queryparam. For most endpoints the maximum page size is 1000, however this might lower depending on the relationships and data that are being requested.
2. Read the response cursor
   - The response may include a `meta.pagination` object. If `meta.pagination.cursors.next` is present and non-null, use that value as `page[cursor]` in your next request.
3. Stop when there's no next cursor
   - If you're on the last page, `meta.pagination.cursors.next` may be null or omitted.

Parameter	Description
page[count]	Maximum number of results returned
page[cursor]	Return results for the cursor provided in a previous response

> ❗ A common gotcha is forgetting to URL-encode the square brackets. For instance, a raw cURL will only work if you replace `[` with `%5B` and `]` with `%5D`.

The `links` field of the response body contains URLs of first, prev, next, and last pages if they exist.

Webhooks

Sample Webhook payload

```
{
  "data": {
    "attributes": {
      "amount_cents": 250,
      "created_at": "2015-05-18T23:50:42+00:00",
      "declined_since": null,
      "patron_pays_fees": false,
      "pledge_cap_cents": null
    },
    "id": "1",
    "relationships": {
      "address": {
        "data": null
      },
      "card": {
        "data": null
      },
      "creator": {
        "data": {
          "id": "3024102",
          "type": "user"
        },
        "links": {
          "related": "https://www.patreon.com/api/user/3024102"
        }
      },
      "patron": {
        "data": {
          "id": "32187",
          "type": "user"
        },
        "links": {
          "related": "https://www.patreon.com/api/user/32187"
        }
      },
      "reward": {
        "data": {
          "id": "599336",
          "type": "reward"
        },
        "links": {
          "related": "https://www.patreon.com/api/rewards/599336"
        }
      }
    },
    "type": "pledge"
  },
  "included": [{ ** * Creator Object ** *
    },
    { ** * Patron Object ** *
    },
    { ** * Reward Object ** *
    },
  ]
}
```


Want webhook functionality without the code? Check out our Zapier plugin.

Webhooks allow you to receive real-time updates from our servers. While there will eventually be many events about which you can be notified, we presently only support webhooks that trigger when you get a new patron, or an existing patron edits or deletes their pledge.

By creating a webhook, you can specify a URL for us to send an HTTP POST to when one of these events occur. This POST request will contain the relevant data from the user action in JSON format. It will also have headers

```
X-Patreon-Event: [trigger]
X-Patreon-Signature: [message signature]
```

where the message signature is the HEX digest of the message body HMAC signed (with MD5) using your webhook's `secret` viewable on the webhooks page. You can use this to verify us as the sender of the message.

> It's important you don't share or expose your webhook secret!

TRIGGERS

A `trigger` is an event type. You can manage webhooks and triggers in Patreon Platform Portal.

> The following webhook triggers have been deprecated and aren't supported. Please use triggers v2.

Trigger	Description
pledge:create **(deprecated)**	Fires when a user pledges to a creator. This trigger fires even if the charge is declined or fraudulent. The pledge object is still created, even if the user is not a valid patron due to charge status.
pledge:update **(deprecated)**	Fires when a user edits their pledge. Notably, the pledge ID will change, because the underlying pledge object is different. The user id should be the primary key to reference.
pledge:delete **(deprecated)**	Fires when a user stops pledging or the pledge is cancelled altogether. Does *not* fire for pledge pausing, as the pledge still exists.

ROBUST RETRIES

To ensure that you can rely exclusively on webhooks data, we've put measures in place to make sure your server does not miss a single event.

In case of failed delivery, perhaps due to network problems or server outages on your end, **we will store the events and make sure none were lost until your server is back up**. Over time, we'll try to send them to you and re-try your server. The next successful call to your server will include all the past webhooks that accumulated.

Our retry schedule is approximately as follows:

- 30 seconds after first failure
- 5 minutes later
- 15 minutes later
- 1 hour later
- 3 hours later
- 1 day later
- 1 week later
- Requires manual retry via our website

You can also use our webhooks page to manually send yourself the queued messages.

PROGRAMMATICALLY ADDING WEBHOOKS

In addition to manually adding webhooks, you can also create, read, update, delete and list webhooks with our API. If you would like to know more please contact us in the developers forum.

External Services

We have an ever expanding set of external services that enable creators to manage and reward patrons in a flexible way.

Zapier

Zapier is an automation tool that connects your favorite apps, such as Gmail, Slack, MailChimp, and over 750 more. You can use the Patreon Zapier plugin to automate repetitive tasks without coding or relying on developers to build the integration.

It's perfect for many of the use cases that would ordinarily require webhooks and has an ever expanding set of use cases. https://zapier.com/zapbook/patreon/

USE CASES

- Track new patrons in Sales Force
- Get notified on slack when a new patron joins or leaves
- Add new patron's contact info to a google sheet
- Sync new patrons with a Mailchimp, Hubspot, ConvertKit, Drip, InfusionSoft, etc.

Our Zapier plugin currently supports pledge activity (adds, updates and deletes). We plan to add more triggers in the future. If you would like to request a trigger, please contact us in the developers forum.

Wordpress

We have a wordpress plugin that allows creators to gate content on their wordpress page to patrons only. For more information check it out.

FAQs

Testing

Testing of the API can be done by creating dummy accounts through the website. To test pledging, it can be useful to:

1. Create a Patreon account with creator page and set it to "per post".
2. Create another Patreon account and use it to pledge to the first account's page. A "per post" pledge will not incur any payments as long as it is cancelled before the end of the month.
3. Use the API as needed.

> Unfortunately, at this time, we do not offer a separate testing/sandbox API.

Accessibility

Patreon is committed to providing accessible websites to all of our users, and to becoming WCAG 2.1 AA-compliant. See our accessibility page for more information.

Patreon encourages developers to follow best practices in designing and building apps that are accessible, and which adhere to the Guidelines established by the Web Accessibility Initiative of the World Wide Web Consortium.

We sometimes link to other sites and provide features built by our users or by third party developers. These parties are not vendors to Patreon, and are not paid by us for their content. We have no control over the accessibility of the content, features, or apps third parties may build using our API, and we are not responsible for their providing inaccessible tools.

Changelog

`2026-03-25` **- V1 CLIENT CREATION RESTRICTION**

Creation of new V1 clients has been restricted. Going forward, please create V2 clients and use the V2 API as V1 API has been deprecated.

`2026-03-11` **- LIVE RESOURCES AND ENDPOINTS**

Live resources, plus `live-access-rules` relationship on the API V2 Campaign resource and CRUD endpoints, have been added. Added new scopes to support reading and writing to lives.

`2026-03-09` **- NAME AND CURRENCY ADDED TO CAMPAIGN RESOURCE**

Two new attributes, `name` and `currency`, have been added to the API V2 campaign resource.

`2025-07-04` **- ADDED EDGE RATE LIMITING**

A higher-level rate limit is enforced when a large number of bad requests are detected within a 10-minute window. Exceeding this threshold will result in a temporary block from the API for 30 minutes.

Please read the Rate Limits documentation for more information on rate limits.

`2024-10-14` **- RATE LIMITS UPDATED**

Per-client and per-token rate limits have been added to the public API.

Please read the Rate Limits documentation for more information on rate limits.